



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

PL-900 Microsoft Power Platform Fundamentals Training

Course Code: C1358

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	21 September 2026	Initial release of the non-WSQ learner guide, derived from the PL-900 courseware.	Dr Alfred Ang

TABLE OF CONTENTS

Introduction.....	4
Course Learning Outcomes.....	4
The Case Study — Contoso Corporation.....	5
Before You Start — Environment Setup.....	5
Topic 01 — Introduction to Microsoft Power Platform.....	6
Lab 1 — Validate the Lab Environment and Tour the Maker Portals.....	7
Lab 2 — Assess the Business Case and Map Connectors to an Integration Need.....	8
Topic 02 — Introduction to Dataverse.....	10
Lab 3 — Build the Contoso Dataverse Data Model.....	10
Lab 4 — Create a Related Table with Copilot and Build a Relationship.....	12
Lab 5 — Environments, Security, DLP and Solution-Based ALM.....	14
Topic 03 — Introduction to Power Apps.....	15
Lab 6 — Build a Canvas App on Dataverse.....	16
Lab 7 — Extend the Canvas App with a New Screen and Power Fx Navigation.....	17
Lab 8 — Build a Model-Driven App with Custom Forms and Views.....	19
Topic 04 — Introduction to Power Automate.....	21
Lab 9 — Build an Automated Cloud Flow with a Condition.....	21
Lab 10 — Test the Flow and Diagnose a Failed Run.....	23
Lab 11 — Approvals, Desktop Flows and Copilot-Assisted Automation.....	24
Topic 05 — Introduction to Power BI.....	26
Lab 12 — Connect, Transform and Model Facilities Data in Power BI.....	26
Lab 13 — Visualise, Publish and Share a Facilities Dashboard.....	28
Topic 06 — Introduction to Power Virtual Agents (Microsoft Copilot Studio).....	29
Lab 14 — Build a Copilot Studio Agent with Topics and Knowledge.....	30
Lab 15 — Add a Tool, Publish the Agent and Resolve Integration Issues.....	31
Exam Focus — What PL-900 Actually Tests.....	33
Exam Preparation.....	35
Glossary.....	35

Introduction

Use this guide alongside the course slides and the lab files in the labs/ folder of the course repository. The slides give you the concepts and the shape of each lab; THIS guide gives you the exact clicks.

Note: Microsoft renamed Power Virtual Agents to Microsoft Copilot Studio. The approved course outline uses the original name; the product you will actually use is Copilot Studio. The concepts — agents, topics, entities, actions and publishing — are unchanged.

Course Learning Outcomes

- LO1: Identify opportunities and assess Power Platform for connecting various devices, databases, software and applications.
- LO2: Utilise Power Platform to integrate data, functions and supporting API-level integration.
- LO3: Perform tests and verify the functioning of Power Platform applications.
- LO4: Highlight and address technical, compatibility or performance issues following the integration of Power Platform.

The Case Study — Contoso Corporation

Contoso needs a single system for employees to submit facilities and maintenance requests — broken equipment, room setup and supply orders — that the facilities team can triage, track and resolve. Each lab builds one part of that solution with a different Power Platform component.

What Contoso needs

- Store facility request and room data in a structured, secure way.
- Let employees submit requests from a phone and let coordinators manage them on a desktop.
- Notify the facilities team automatically when a high-priority request arrives.
- Report on request volume, cost and turnaround time for management.
- Answer common facilities questions with a self-service agent.
- Govern the whole thing with environments, DLP policies and solution-based ALM.

Every lab in this course builds one part of this single solution. By the end of Day 2 you will have a Dataverse data model, a canvas app, a model-driven app, two cloud flows, a Power BI dashboard and a published Copilot Studio agent — all working on the same data.

Before You Start — Environment Setup

What you need

- A Windows laptop with a modern browser — Microsoft Edge or Google Chrome are recommended.

- A Microsoft 365 work or school account with Power Platform access. A trial or a free Power Apps developer environment is acceptable.
- A Power Platform environment with Microsoft Dataverse provisioned, and maker-level permissions in it.
- Power BI Desktop installed (free from the Microsoft Store or powerbi.microsoft.com/desktop) for Topic 5.
- The lab files and mock data from the course repository:
<https://github.com/tertiarycourses/C1358-PL-900-Microsoft-Power-Platform-Fundamentals-Training>

Get a free environment if you do not have one

If your organisation has not given you a Power Platform environment, join the free Power Apps Developer Plan. It provides a personal environment with Dataverse, Power Automate and Copilot Studio at no cost — everything these labs need.

Developer Plan sign-up: <https://aka.ms/PowerAppsDevPlan>

Maker portal: <https://make.powerapps.com>

Power Automate: <https://make.powerautomate.com>

Power BI service: <https://app.powerbi.com>

Copilot Studio: <https://copilotstudio.microsoft.com>

Admin center: <https://admin.powerplatform.microsoft.com>

The single most important setup rule

Every portal keeps its OWN last-used environment. Before you build anything in a lab, check the environment picker at the top right of the page and make sure it names the same environment you used in Lab 1. Most 'my table has disappeared' problems are simply the wrong environment.

Conventions used in every lab

- Menu paths and button names are shown in bold in the slides and in plain text here — for example: select + New table, then Table (advanced properties).
- Power Fx formulas and other values you type are shown in a code block.
- Mock data for each lab is in labs/<lab folder>/data/ in the course repository.
- Each lab ends with a Test it check, a troubleshooting table, and checkpoint questions.
- Labs build on each other within the Contoso scenario, so complete them in order.

Topic 01 — Introduction to Microsoft Power Platform

Business value · connectors · DLP, compliance, privacy and accessibility (LO1 · K1, K3 · A1, A2)

Key concepts

- What Power Platform is — A low-code suite — Power Apps, Power Automate, Power BI, Copilot Studio and Power Pages — sitting on Dataverse and 1,400+ connectors so business users build solutions without writing traditional code.
- Why it exists — Changing workforce expectations, the rising cost of custom development, and the need to be agile and scale delivery without growing the developer headcount.
- Middleware role (K1) — Power Platform acts as integration middleware: connectors, Dataverse and flows sit between systems that were never designed to talk to each other.
- Platform types (K3) — Solutions run across web, mobile, desktop and Teams — the same app and data model surface on every platform without a rewrite.
- Connectors — Standard, premium and custom connectors expose an API as a set of triggers and actions; a custom connector wraps any REST API you own.
- Governance from day one — DLP policies, environment strategy, compliance, data privacy and accessibility guardrails must be designed in, not bolted on.

Lab 1 — Validate the Lab Environment and Tour the Maker Portals

Learning objective: you will be able to sign in to the Power Platform, confirm the environment and licence, and navigate the four maker portals used throughout the course.

Estimated duration: 45 minutes.

Scenario: Before building anything you confirm you have a working environment. You sign in to the Power Apps maker portal, verify which environment you are in, check your licence, and tour the Power Automate, Power BI and Copilot Studio portals so you know where each service lives.

What you will build

A validated Power Platform environment plus a portal reference sheet recording your environment name, region, licence type and the Dataverse state. (Tools used: Power Apps maker portal, Power Automate, Power BI service, Copilot Studio, Power Platform admin center.)

Step-by-step

1. Open a private/InPrivate browser window and go to the Power Apps maker portal, then sign in with your Microsoft 365 work or school account.

<https://make.powerapps.com>

2. Read the environment picker at the top right. Note the environment NAME and REGION — every lab in this course must be built in this same environment.
3. If a Dataverse database has not been provisioned, select the environment picker and choose an environment whose Type is 'Developer' or 'Trial (subscription)'. A developer environment is free and includes Dataverse.
4. Select the gear icon, then Session details. Record the Environment ID, the Tenant ID and the Power Apps version on your reference sheet.
5. In the left navigation select Tables, then switch the filter from Default to All. Confirm you can see the standard tables such as Account and Contact — this proves Dataverse is live in this environment.

6. Open the Power Automate portal in a new tab and confirm the SAME environment is selected in the environment picker.

<https://make.powerautomate.com>

7. Open the Power BI service in a third tab and confirm you can reach My workspace.

<https://app.powerbi.com>

8. Open Copilot Studio in a fourth tab and confirm the same environment is selected.

<https://copilotstudio.microsoft.com>

9. Open the Power Platform admin center and select Environments. Note which environments you can see and what your role is — this is the governance view.

<https://admin.powerplatform.microsoft.com>

10. Complete the portal reference sheet: for each of the five portals record its URL, what you build there, and who in an organisation typically uses it.

Test it

You can open all five portals signed in as the same user, the SAME environment name appears in the picker on the Power Apps, Power Automate and Copilot Studio portals, and Tables shows the standard Dataverse tables.

If it doesn't work

- No environment with Dataverse — Create a free developer environment at <https://aka.ms/PowerAppsDevPlan>, then re-check the environment picker.
- 'You need permission' on the admin center — You are not an environment or tenant admin. This is expected for a maker account — record what you CAN see and continue.
- Wrong environment selected — Every portal keeps its own last-used environment. Set the picker explicitly in each tab before you build anything.

Checkpoint questions

- Which portal do you use to build a canvas app, and which to set a DLP policy?
- Why must every lab be built in the same environment?
- What does it mean if Tables shows no tables at all?

Lab 2 — Assess the Business Case and Map Connectors to an Integration Need

Learning objective: you will be able to identify integration opportunities across Contoso's systems and perform a feasibility assessment that selects the right connector for each need.

Estimated duration: 60 minutes.

Scenario: Contoso's facilities process is spread across email, a spreadsheet and a legacy web system. You analyse the current state, identify where Power Platform can create connections between those systems, and perform a feasibility scan that decides which connector — standard, premium or custom — serves each requirement.

What you will build

A completed integration opportunity register and a connector feasibility assessment recommending a connector and licence tier for each of Contoso's six integration needs. (Tools used: Power Platform connector reference, Power Apps maker portal, Microsoft Learn documentation.)

Mock data for this lab

- labs/lab-02-business-value-connectors/data/contoso-current-state.csv
- labs/lab-02-business-value-connectors/data/connector-feasibility-template.csv

Step-by-step

11. Open labs/lab-02-business-value-connectors/data/contoso-current-state.csv. It lists how Contoso handles facilities requests today, system by system.
12. For each row, write down the PAIN it causes — rekeying, delay, no audit trail, no visibility. This is the evidence for the business case.
13. Complete the integration opportunity register: for each pain point, record the two systems that need to be connected, the data that must move between them, and the direction of flow. This is ability A1.
14. In the maker portal, select Data then Connectors. Filter the list and search for each system named in the CSV: Outlook, SharePoint, Excel, SQL Server, Teams.
15. For each connector, open it and record whether it is Standard or Premium, and list two triggers and two actions it exposes. Those triggers and actions ARE the API surface (K5).
16. Search the connector list for Contoso's legacy 'Building Services' REST API. It is not there — note that this need requires a CUSTOM connector built from an OpenAPI definition, a Postman collection or from blank.
17. Complete the feasibility assessment in connector-feasibility-template.csv. For each need score: is a connector available, is it standard or premium, does it need a gateway, and what is the licence impact. This is ability A2.
18. Write a one-paragraph recommendation naming the connector for each of the six needs and flagging which two require premium licensing.
19. Classify each of Contoso's target platforms — web, mobile, desktop and Teams — and note that one Power Platform solution serves all four without a rewrite (K3).
20. Review the DLP implication: if a policy puts SharePoint in Business and the custom connector in Non-business, the two cannot be used in the same app or flow.

Test it

Your register names both systems and the data direction for every opportunity, and your feasibility assessment recommends a specific connector with its licence tier for all six needs, including a custom connector for the legacy REST API.

If it doesn't work

- Cannot find a connector in the maker portal — Use <https://learn.microsoft.com/connectors/connector-reference/> — it lists every connector and its tier without needing a licence.
- Unsure whether a connector is premium — The connector card shows a PREMIUM badge. Premium requires a per-user or per-app Power Apps/Power Automate licence.
- No API for a legacy system — If it exposes no API at all, the integration path is a desktop flow (RPA) driving the UI, not a connector.

Checkpoint questions

- What makes Power Platform 'middleware' in this scenario?
- Name the three ways to create a custom connector.
- Why does a DLP policy break an app that uses two connectors from different groups?

Topic 02 — Introduction to Dataverse

Tables, columns, relationships · environments · business logic · administration (LO1 · K1 · A2)

Key concepts

- What Dataverse is — A secure, cloud-hosted business data platform — not just a database. It adds metadata, role-based security, business logic and auditing on top of storage.
- Beyond a spreadsheet — Enforced data types, relationships, row/column-level security, server-side validation, views, forms and change tracking — none of which a workbook gives you.
- Tables, columns, rows — Tables define the entity, columns define the fields and their data types, and rows hold the records. Standard, custom and managed (virtual) tables coexist.
- Relationships — One-to-many, many-to-one and many-to-many relationships connect tables; a lookup column is how a one-to-many relationship appears on a form.
- Business logic — Business rules (no-code), Power Fx formula columns, real-time workflows, cloud flows and low-code plug-ins — pick the lightest tool that does the job.
- Environments and ALM — Environments isolate dev, test and production; solutions package the work and Power Platform pipelines move it between them.

Lab 3 — Build the Contoso Dataverse Data Model

Learning objective: you will be able to create a Dataverse table with the full range of column types and populate it with business data.

Estimated duration: 60 minutes.

Scenario: Contoso needs a central store for facilities request data. You create the Facility Request table in Dataverse, add text, date, currency and choice columns, and load the sample request data so later labs have something to display.

What you will build

A Facility Request table in Dataverse with seven columns — including three choice columns — and eight sample request rows. (Tools used: Power Apps maker portal, Microsoft Dataverse.)

Mock data for this lab

- labs/lab-03-dataverse-data-model/data/facility-requests.csv

Step-by-step

21. Go to the maker portal, confirm your course environment is selected, and in the left navigation select Tables.

<https://make.powerapps.com>

22. Select the + New table dropdown and choose Table (advanced properties).

23. In the Properties panel set the Display name to 'Facility Request'. The plural name auto-populates as Facility Requests.

24. Select the Primary Column tab and set its Display name to 'Request Title', then select Save. The primary column is the human-readable name of every row.

25. Under 'Facility Request columns and data' select +, and add: Display name 'Description', Data type Multiple Lines of Text. Expand Advanced options and set Maximum character count to 2000. Save.

26. Add a second column: Display name 'Date Requested', Data type Date and Time, Format Date Only. Expand Advanced options and set Time Zone adjustment to User Local. Save.

27. Add a third column: Display name 'Estimated Cost', Data type Currency. Leave the defaults and Save. Currency columns store a base and a transaction value.

28. Add the 'Category' choice column. Set Data type to Choice, set 'Sync with global choice?' to No, and add the labels Maintenance, Equipment, Supplies, Room Setup and Other. Set Default Choice to None. Save.

29. Add the 'Priority' choice column with these labels AND values — Low 1, Medium 2, High 3, Urgent 4. Set Default Choice to None and Save. Record these integers: Power Automate compares against the VALUE, not the label.

30. Add the 'Status' choice column with labels New, In Progress, Completed and Cancelled. Set Default Choice to New and Save.

31. Select Edit on the table, then + New row, and enter the eight rows from labs/lab-03-dataverse-data-model/data/facility-requests.csv, filling Request Title, Description, Category, Priority, Status, Date Requested and Estimated Cost.

32. Switch the view to the grid and confirm all eight rows display with the correct choice labels rendered as text, not numbers.

Test it

The Facility Request table lists eight rows; Category, Priority and Status render as choice labels; and Priority values map to Low=1, Medium=2, High=3, Urgent=4.

If it doesn't work

- Table (advanced properties) is missing — You are on the classic Tables view. Use + New table > Table (advanced properties); if unavailable, create the table then rename the primary column afterwards.
- Choice values are not 1–4 — Open the Priority column, select Edit choices, and correct the Value box next to each label. Lab 8's condition depends on High=3 and Urgent=4.
- Currency column will not save — The environment needs a base currency. Open the admin center > Environment > Settings > Product > Currencies and confirm one is set.

Checkpoint questions

- Name three things Dataverse gives you that a spreadsheet does not.
- What is the primary column used for?
- Why does a choice column store an integer value as well as a label?

Lab 4 — Create a Related Table with Copilot and Build a Relationship

Learning objective: you will be able to use AI to create and edit tables and columns, and connect two tables with a lookup relationship.

Estimated duration: 45 minutes.

Scenario: Contoso's requests need to be tied to a specific room. You create a Room table using the Copilot pane instead of the form designer, then add a lookup column on Facility Request so every request points at one room — a one-to-many relationship.

What you will build

A Room table with five columns and three sample rows, plus a Room lookup column on Facility Request creating a one-to-many relationship. (Tools used: Power Apps maker portal, Microsoft Dataverse, Copilot in Dataverse.)

Mock data for this lab

- labs/lab-04-copilot-table-relationship/data/rooms.csv

Step-by-step

33. In the maker portal select Tables, then 'Start with a blank table'.
34. Rename the table from Table1 to 'Room'.
35. In the Copilot pane, rename the default column by entering: Rename New Column in the Room table to Room Name
36. In the Copilot pane, add three text columns at once by entering: Add new columns named Campus, Building, and Floor to the Room table
37. In the Copilot pane, add the yes/no column by entering: Add a new yes/no column named Conference Room to the Room table

38. Review what Copilot produced. Copilot results vary — check each column's data type and correct any that are wrong before saving. This review step is the maker's job.
39. Add the three sample rows from labs/lab-04-copilot-table-relationship/data/rooms.csv — 301 A / North / HighPoint / 3 / Yes, 233 / South / Sierra / 2 / No, and 401 B / East / Jacobson / 4 / Yes.
40. Select Save and exit, then in the 'Done working?' dialog select Save and exit.
41. Return to Tables, select All, search for 'Facility', and open the Facility Request table.
42. Under Schema select Columns, then + New column. Set Display name 'Room', Data type Lookup, and Related table 'Room'. Save.
43. Open the Facility Request rows and set the Room value on at least three requests, confirming the lookup offers only the three rooms you created.
44. Open the Room table and select Relationships. Confirm a one-to-many relationship from Room to Facility Request now exists.

Test it

The Room table holds three rows, Facility Request has a working Room lookup that resolves to a room name, and the Relationships tab shows the one-to-many link.

If it doesn't work

- Copilot pane is not visible — Copilot may be disabled for the environment or region. Add the columns manually with + New column — the learning outcome is the data model, not the tool.
- Copilot created the wrong data type — Expected. Open the column, correct the type, and note the correction — reviewing AI output is part of the exam objective.
- Lookup shows no rooms — The Room rows were not saved. Reopen the Room table, confirm the rows persisted, then refresh the Facility Request form.

Checkpoint questions

- What kind of relationship does a lookup column create?
- Why must a maker review what Copilot generates?
- Which table holds the lookup column in a one-to-many relationship?

Lab 5 — Environments, Security, DLP and Solution-Based ALM

Learning objective: you will be able to describe environments, the security model, data policies and how solutions and pipelines move work between environments.

Estimated duration: 60 minutes.

Scenario: Contoso must govern what makers can build. You examine the environment list, inspect security roles on the Dataverse table you built, design a DLP policy that separates business from non-business connectors, and package your work into a solution so it can be promoted from development to production.

What you will build

A governance pack: an environment strategy table, a security role mapping for the Facility Request table, a two-group DLP policy design, and an exported unmanaged solution containing your tables. (Tools used: Power Platform admin center, Power Apps maker portal, Dataverse security roles, solutions and pipelines.)

Mock data for this lab

- labs/lab-05-governance-alm/data/dlp-policy-design.csv

Step-by-step

45. Open the Power Platform admin center and select Environments. Record the name, type and region of every environment you can see.

<https://admin.powerplatform.microsoft.com>

46. Complete the environment strategy table: state what Default, Developer, Sandbox and Production environments are each FOR, and why production work must never be built in Default.
47. In the maker portal open the Facility Request table, select the Settings gear then Advanced, and review the security roles that grant access to it.
48. Map the Dataverse security model on paper: environment access, then security role, then table privilege (Create/Read/Write/Delete), then row-level access scope (User / Business Unit / Organization).
49. Decide which Contoso persona gets which scope — an employee sees only their own requests (User), a coordinator sees the whole organisation (Organization).
50. In the admin center select Policies then Data policies. Review any existing policy and note its Business, Non-business and Blocked groups.
51. Complete labs/lab-05-governance-alm/data/dlp-policy-design.csv: place Dataverse, Office 365 Outlook, SharePoint and Teams in Business, and place consumer connectors such as Twitter/X, Dropbox and Facebook in Non-business.
52. Write down the consequence: an app or flow may not combine a Business connector with a Non-business one. Explain how that stops data leaving Contoso.
53. In the maker portal select Solutions, then + New solution. Name it 'Contoso Facilities', set the publisher, and create it.
54. Select Add existing > Table and add both Facility Request and Room, choosing to include all components.
55. Select Export solution, choose Unmanaged, and download the .zip. Note that unmanaged is for source control and managed is for production.
56. Describe the ALM path in your own words: Dev environment (unmanaged) → solution or Power Platform pipeline → Test → Production (managed), with no manual rebuild.

Test it

You can name the purpose of each environment type, state the row-level scope for both Contoso personas, explain what your DLP policy blocks, and you hold an exported unmanaged solution .zip containing both tables.

If it doesn't work

- Data policies is greyed out — DLP policies need tenant or environment admin rights. Complete the design on paper — the exam objective is to DESCRIBE the policy, not to create one.
- Solution export fails — A component is still publishing. Select Publish all customizations on the solution, wait for it to finish, then export again.
- Cannot see other environments — You only see environments you have a role in. Record what is visible and note why the list is short.

Checkpoint questions

- Why should you never build a production app in the Default environment?
- What is the difference between a managed and an unmanaged solution?
- How does a DLP policy actually prevent data exfiltration?

Topic 03 — Introduction to Power Apps

Canvas apps · model-driven apps · Copilot-assisted app creation (LO2 · K5 · A3)

Key concepts

- What Power Apps is — A low-code app platform for building business applications that run in a browser and on mobile, over Dataverse or any connector.
- Canvas apps — Pixel-precise, designer-driven apps. You start from the screen, drag controls onto the canvas and bind them with Power Fx — best for task-focused mobile apps.
- Model-driven apps — Data-first apps generated from the Dataverse model. Forms, views and navigation come from the schema — best for complex, process-heavy desktop work.
- Power Fx (K5) — The Excel-like formula language behind canvas apps; functions such as SubmitForm, Navigate, Patch and Filter call the underlying data APIs for you.
- Copilot-assisted creation — Describe the app in natural language and Copilot drafts the tables, screens and logic — the maker then reviews and refines the result.
- Plan designer and code apps — Plan designer turns a business problem into a proposed solution set; code apps let pro developers bring their own React/TypeScript UI onto Power Platform.

Lab 6 — Build a Canvas App on Dataverse

Learning objective: you will be able to build a canvas app connected to a Dataverse table and customise its screens, galleries and forms.

Estimated duration: 60 minutes.

Scenario: Contoso employees need a mobile-friendly way to see and submit facilities requests. You generate a three-screen canvas app from the Facility Request table, then customise the header with the signed-in user's name and format the gallery so it shows the information a coordinator actually needs.

What you will build

A published canvas app named 'Facility Request App' with a browse gallery, a detail screen, an edit form and a personalised welcome header. (Tools used: Power Apps Studio, Microsoft Dataverse, Power Fx.)

Step-by-step

57. In the maker portal select + Create, then under 'Start from data' select Dataverse.

<https://make.powerapps.com>

58. In the Search field enter 'Facility', select the Facility Requests table, and choose Create app. Power Apps generates a three-screen app.

59. If the 'Welcome to Power Apps Studio' screen appears, tick 'Don't show me this again' and select Skip.

60. Explore the Tree view. Identify the gallery on the browse screen, the display form on the detail screen and the edit form on the edit screen — the three controls every generated app is built from.

61. Select the Facility Requests header on the default screen, then + Insert > Text label.

62. Set the label's Text property to the literal string "Welcome" and format it: Font size 16, Font colour White, Fill Blue, Align Right, Height 52.

"Welcome"

63. Insert a second Text label and set its Text property to the Power Fx expression below, which reads the signed-in user from the Office 365 Users service.

`User().FullName`

64. Format the second label to match: Font size 16, Font colour White, Fill Blue, Align Right, Height 52, Width 225.

65. Select the gallery and change its Layout so the title shows Request Title and the subtitle shows Status — make the list scannable.

66. Select the Play button to preview. Confirm the gallery lists your eight rows and your own name appears in the header.

67. Select a row and confirm the detail screen shows that request's values, then select the edit icon and confirm the form is editable.

68. Select Save, name the app 'Facility Request App', then select Publish and 'Publish this version'.

Test it

The published app opens in Play mode, the gallery lists all eight Facility Request rows, your full name renders in the header, and selecting a row opens its details.

If it doesn't work

- `User().FullName` shows blank — The app needs the Office 365 Users connection. Select Data > + Add data > Office 365 Users, then re-run the preview.
- Gallery is empty — The app is pointed at an empty table or the wrong environment. Confirm the environment picker matches Lab 3 and that the table still holds its rows.
- Cannot find the Text property — Select the control first, then choose Text in the property dropdown at the top left of the formula bar.

Checkpoint questions

- What are the three screens a generated canvas app creates?
- What does `SubmitForm()` do that saves you writing an API call?
- When would you choose a canvas app over a model-driven app?

Lab 7 — Extend the Canvas App with a New Screen and Power Fx Navigation

Learning objective: you will be able to add a screen and a form to a canvas app and wire navigation with Power Fx functions that call the underlying data API.

Estimated duration: 60 minutes.

Scenario: Coordinators also need to add rooms without leaving the app. You add a second screen containing an edit form bound to the Room table, then use the Power Fx functions `NewForm`, `SubmitForm` and `Navigate` to move between screens and commit the record — API-level integration expressed as formulas.

What you will build

A second screen, 'New Room Screen', with a working Room edit form and two buttons that navigate between the request list and the room form. (Tools used: Power Apps Studio, Microsoft Dataverse, Power Fx.)

Step-by-step

69. Open Facility Request App in Power Apps Studio and select Edit.
70. From the command bar select New screen, then choose the Header and Footer layout.
71. In Tree view rename the new screen from Screen1 to 'New Room Screen'.
72. Select + in the header container, insert a Text label, and set its Text property to "Add New Room". Format it white on blue, right aligned, height 52.

"Add New Room"

73. Select the Header container itself and set its Fill to Blue so the two screens match.

74. In the main container select + Insert > Edit form. In the data Search field enter 'Room' and select the Rooms table.
75. If the form shows system columns such as Import Sequence Number, Time Zone Rule Version Number or Record Created On, select each in Tree view and delete it.
76. In the form's Properties pane set Default mode to New so the form opens blank ready for data entry.
77. Select the footer, insert a Button, set its Text to "Submit", and set its OnSelect property to the formula below — SubmitForm writes the row to Dataverse, then Navigate returns to the list.

```
SubmitForm(Form2); Navigate('Facility Requests screen')
```

78. Return to the Facility Requests screen, select the gallery, and from the command bar insert a Button. Set its Text to "New Room".

"New Room"

79. Set that button's OnSelect to the formula below — NewForm resets the form to a blank record before navigating, so you never edit the previous row by accident.

```
NewForm(Form2); Navigate('New Room Screen')
```

80. Select Play, choose New Room, add a fourth room (for example 512 / West / Kepler / 5 / Yes), select Submit, and confirm you land back on the request list.
81. Open the Room table in the maker portal and confirm the new room was written to Dataverse. Save and Publish the app.

Test it

Selecting New Room opens a blank Room form; Submit writes the row to the Dataverse Room table and returns you to the request list; the new room is visible in the maker portal.

If it doesn't work

- Submit does nothing — The formula names the wrong form. Check the form's actual name in Tree view — it may be Form1 rather than Form2 — and update SubmitForm() to match.
- The form saves over an existing room — NewForm() is missing from the navigation button. Without it the form keeps the last selected record and SubmitForm updates rather than creates.
- Navigate reports the screen does not exist — Screen names are literal. Use the exact name from Tree view, in single quotes if it contains spaces.

Checkpoint questions

- What is the difference between NewForm() and EditForm()?
- Which Power Fx function commits the form's data to Dataverse?
- How does a Power Fx function relate to an API call?

Lab 8 — Build a Model-Driven App with Custom Forms and Views

Learning objective: you will be able to build a model-driven app from the Dataverse model, customise its form and view, and publish it.

Estimated duration: 60 minutes.

Scenario: The facilities team manages requests all day on a desktop, so they need a full-featured app rather than a mobile canvas. You build a model-driven app using the modern designer, group its navigation, then customise the Facility Request main form and the Active Facility Requests view so the columns the team needs are visible.

What you will build

A published model-driven app, 'Contoso Facilities Management', with grouped navigation, a customised main form and a customised public view. (Tools used: Power Apps modern app designer, Microsoft Dataverse, forms and views.)

Step-by-step

82. In the maker portal select + Create, then choose 'Blank Page with navigation'. Name the app 'Contoso Facilities Management' and select Create.

<https://make.powerapps.com>

83. In the App Designer select + Add page, choose Dataverse table, and select the Facility Request, Room, Account and Contact tables.

84. Ensure 'Show in navigation' is ticked, then select Add.

85. On the left select Pages. Under Navigation select New group and rename it to 'Customers'.

86. Select the three dots beside the Customers group, choose New group, and rename that one to 'Facilities'.

87. Select the Facility Requests view and use Move down until it sits inside the Facilities group. Repeat for the Rooms view. Save.

88. In a new tab open Tables, search for Facility Request, open it, and under Data experiences select Forms.

89. Open the Information form whose Form type is Main, using Commands (...) > Edit > Edit in new tab.

90. Drag Owner and Status into the Header area — header fields stay visible as the user scrolls.

91. Drag Description, Date Requested, Estimated Cost, Category and Priority into the body in that order, then select Save and publish and close the designer.

92. Back on the table, under Data experiences select Views, and edit the 'Active Facility Requests' view.

93. Remove the Created On column, then add Description, Date Requested, Estimated Cost, Category, Priority and Status. Save and publish, then close.

94. Return to Apps, open Contoso Facilities Management, select Save and Publish, then Play.

95. Verify the navigation shows the Facilities group, the list view shows your columns, and opening a record shows the form layout you built.

Test it

The published app opens with grouped navigation; the Active Facility Requests view shows the six added columns; and a record's form shows Owner and Status in the header with the other fields in the order you set.

If it doesn't work

- Changes do not appear in the app — Forms and views must be published separately from the app. Select Save and publish in the form/view designer, then Save and Publish the app.
- Move down will not place the view in the group — Groups only accept pages below them. Keep selecting Move down until the item indents under the group heading.
- The table is missing from + Add page — You are in the wrong environment. Check the environment picker matches Lab 3.

Checkpoint questions

- Where does a model-driven app get its user interface from?
- Name two things a model-driven app gives you for free that you must build by hand in a canvas app.
- Why must you publish a form after editing it?

Topic 04 — Introduction to Power Automate

Cloud, desktop and business process flows · triggers, actions, approvals (LO2 · K5 · A4)

Key concepts

- What Power Automate is — A workflow engine that connects services and automates repetitive work, from an email notification to a full RPA robot driving a legacy desktop app.
- Three flow types — Cloud flows run in the service; desktop flows (RPA) drive the Windows UI; business process flows guide a person through a staged process.
- Triggers and actions (K5) — A trigger starts the flow — automated, instant or scheduled — and each action calls a connector operation, which is an API call under the hood.
- Conditions and loops — Condition, Switch, Apply to each and Do Until add branching and iteration; expressions transform values in transit.
- Approvals — The Approvals connector sends an actionable request to Teams or Outlook and waits for the outcome before continuing the flow.
- Copilot in Power Automate — Describe the automation in words and Copilot builds the trigger, condition and actions, which you then verify and test.

Lab 9 — Build an Automated Cloud Flow with a Condition

Learning objective: you will be able to create an automated cloud flow triggered by a Dataverse event, add a condition and send a notification through a connector.

Estimated duration: 75 minutes.

Scenario: Contoso wants the facilities team alerted the moment a high-priority request arrives. You build an automated cloud flow that triggers when a row is added to Facility Request, retrieves the full row, tests the Priority value, and sends an Outlook email only when the request is High or Urgent.

What you will build

A saved cloud flow, 'Notify on High Priority Request', with a Dataverse trigger, a Get a row by ID action, an Or condition on Priority and a Send an email action. (Tools used: Power Automate, Microsoft Dataverse connector, Office 365 Outlook connector.)

Step-by-step

96. Open the Power Automate maker portal and confirm the environment picker shows your course environment.

<https://make.powerautomate.com>

97. Select + Create, then Automated cloud flow. Name it 'Notify on High Priority Request'.

98. In the trigger search box enter 'When a row is added' and select 'When a row is added, modified or deleted (Microsoft Dataverse)'. Select Create.

99. If the trigger reports Invalid Parameters, select Change connection reference > Add new, name the connection 'Dataverse', set Authentication type OAuth, sign in, and select your account.

100. Configure the trigger: Change type = Added, Table name = Facility Requests, Scope = Organization so it fires for every user, not just you.

101. Below the trigger select + to add an action. Search for 'Get a row by ID' and select it under Microsoft Dataverse.

102. Configure it: Table name = Facility Requests, and set Row ID from Dynamic content to the trigger's Facility Request value. The trigger returns only the row's ID, so this action fetches the full record.

103. Add a Condition below the action. Set the left box from Dynamic content to Priority (under Get a row by ID), the operator to 'is equal to', and the right box to 3 — the stored value for High.

3

104. Add a second row to the condition for Urgent, using the value 4, and change the And dropdown to Or. The condition now reads Priority = 3 OR Priority = 4.

4

105. In the If yes branch select + > Add an action, search 'Send an email' and select Send an email (V2) from Office 365 Outlook. Sign in if prompted.

106. Configure the email: To = your own address; Subject = 'High Priority Facility Request:' followed by the Request Title from Dynamic content.

High Priority Facility Request: @{Request Title}

107. Set the Body to a short sentence plus the Category, Priority and Description dynamic values from Get a row by ID, each on its own line. Leave the If no branch empty.
108. Select Save. Read the trigger, condition and action back as a sentence to confirm the logic before testing.

Test it

The flow saves with no validation errors, the condition uses Or across Priority = 3 and Priority = 4, and the Send an email action sits in the If yes branch only.

If it doesn't work

- Trigger shows Invalid Parameters — The Dataverse connection reference is missing. Select Change connection reference > Add new and sign in again.
- Priority compares as text and never matches — Use the numeric Value, not the label. High is 3 and Urgent is 4 — the integers you set in Lab 3.
- Dynamic content list is empty — The Get a row by ID action has not been saved. Save the flow, reopen it, and the dynamic tokens will appear.

Checkpoint questions

- What is the difference between a trigger and an action?
- Why is a Get a row by ID action needed after the Dataverse trigger?
- Name the three types of cloud flow.

Lab 10 — Test the Flow and Diagnose a Failed Run

Learning objective: you will be able to test a flow end to end, read the run history, and diagnose and resolve a failing connection between two application programs.

Estimated duration: 60 minutes.

Scenario: A flow that saves is not a flow that works. You trigger the flow with real data, read the run history step by step, then deliberately break the integration and diagnose it from the error — the testing and verification skills the TSC requires.

What you will build

A successful flow run with an email received, plus a fault log recording one induced failure, the error message and the fix applied. (Tools used: Power Automate run history, Microsoft Dataverse, Office 365 Outlook.)

Mock data for this lab

- labs/lab-10-flow-testing-diagnostics/data/fault-log-template.csv

Step-by-step

109. Open the Facility Request table (or the model-driven app from Lab 8) and add a new row with Priority set to High.
110. In Power Automate select My flows, open your flow, and look at the 28-day run history. Confirm a run appears with the status Succeeded.
111. Open the run and expand every step. Record the inputs and outputs of the trigger, Get a row by ID, and the condition — note the condition evaluated to true.
112. Check your inbox and confirm the notification arrived with the correct request title, category, priority and description.
113. Now test the negative path: add a second row with Priority set to Low. Confirm the flow ran, the condition evaluated FALSE, and no email was sent. A flow that correctly does nothing is still a passing test (A6).
114. Induce a fault. Edit the Send an email action and clear the To field, then Save.
115. Add another High priority row and let the flow run. Open the failed run in the history and read the error message on the red step.
116. Record the failure in labs/lab-10-flow-testing-diagnostics/data/fault-log-template.csv: the symptom, the step that failed, the exact error text, the root cause and the fix (A7).
117. Restore the To address, Save, and use Resubmit on the failed run rather than creating a new row — this re-runs the same trigger data (A8).
118. Confirm the resubmitted run now succeeds and the email arrives.
119. Induce a second, different fault of your choosing — for example compare Priority against the label 'High' instead of 3 — and log its symptom, cause and fix.
120. Write a two-sentence verification statement: what you tested, and what evidence proves the integration between Dataverse and Outlook works.

Test it

Your run history shows at least one Succeeded run that sent an email, one run where the condition correctly evaluated false, and one Failed run that you diagnosed, fixed and successfully resubmitted.

If it doesn't work

- No run appears at all — The trigger scope is User rather than Organization, or the row was added in a different environment. Check both.
- Run succeeded but no email arrived — Check the junk folder, then confirm the To address in the action's run inputs is the address you actually monitor.
- Resubmit is greyed out — Resubmit only applies to a failed or cancelled run. Open the run itself, not the flow, to find the button.

Checkpoint questions

- Where do you look to find out exactly why a flow failed?
- Why is a run where the condition is false still a valid test result?
- What does Resubmit do that adding a new row does not?

Lab 11 — Approvals, Desktop Flows and Copilot-Assisted Automation

Learning objective: you will be able to describe approvals, Teams/Outlook/SharePoint automation, desktop flows (RPA) and using AI to create and modify flows.

Estimated duration: 75 minutes.

Scenario: Not every process is a notification. You extend the solution with an approval that pauses the flow until a manager responds, examine where desktop flows fit when a legacy system has no API, and use Copilot to draft a flow from a plain-English description — then review what it produced.

What you will build

An approval flow for high-cost requests, a desktop-flow scenario assessment, and a Copilot-generated flow that you have reviewed and corrected. (Tools used: Power Automate, Approvals connector, Microsoft Teams, Power Automate for desktop, Copilot in Power Automate.)

Step-by-step

121. Create a second automated cloud flow named 'Approve High Cost Request', triggered by a row added to Facility Requests, with a Get a row by ID action as before.
122. Add a condition testing whether Estimated Cost is greater than 500.

500

123. In the If yes branch add the action 'Start and wait for an approval'. Set Approval type to 'Approve/Reject - First to respond'.
124. Set the Title to include the Request Title, set Assigned to your own address, and put the cost and description in Details. Save.
125. Add a condition after the approval testing whether the Outcome is equal to Approve.

Approve

126. In the If yes branch add a Dataverse 'Update a row' action that sets Status to In Progress; in the If no branch set Status to Cancelled.
127. Test it: add a request with an Estimated Cost above 500, respond to the approval in Outlook or Teams, and confirm the row's Status changed accordingly.
128. Now assess desktop flows. Contoso's legacy Building Services system is a Windows desktop application with no API. Write down why a cloud flow cannot integrate with it and why a desktop flow (RPA) can.
129. List three desktop-flow use cases: driving a legacy application, automating a website with no API, and working with terminal emulation software.
130. Compare attended and unattended desktop flows — who is signed in, when each runs, and what licence each needs.
131. Use Copilot: create a new flow and describe it in words, for example 'When a new facility request is added and the category is Equipment, post a message to Teams'.

132. Review the flow Copilot produced. Check the trigger, the condition and the action, correct anything wrong, and record two things you had to change. AI drafts, the maker verifies.

Test it

The approval flow pauses until you respond and then updates the Dataverse row's Status to match your decision; you can state when a desktop flow is required instead of a cloud flow; and you have reviewed and corrected a Copilot-drafted flow.

If it doesn't work

- The approval never arrives — Assigned to must be a full email address on the same tenant. Check the run history inputs for the approval step.
- The flow waits forever — 'Start and wait for an approval' blocks by design until someone responds. Respond in Outlook, Teams or the Power Automate Approvals page.
- Copilot builds the wrong trigger — Expected. Delete the trigger, add the right one manually, and note the correction — reviewing AI output is an exam objective.

Checkpoint questions

- What does 'Start and wait for an approval' do to the flow's execution?
- When must you use a desktop flow instead of a cloud flow?
- Why does a Copilot-generated flow still need a maker to review it?

Topic 05 — Introduction to Power BI

Data modelling · transformation · visualisation · dashboards and sharing (LO3 · K2 · A5, A6)

Key concepts

- What Power BI is — The analytics member of the Power Platform: connect to data, model it, visualise it and share the result as reports and dashboards.
- The parts of Power BI — Power BI Desktop authors the model and report; the Power BI service publishes, shares and refreshes it; the mobile apps consume it.
- Get and transform — Power Query cleans and reshapes data — remove columns, change types, split, merge and append — before it ever reaches the model.
- Model and visualise — Relationships join tables into a model; visuals, slicers and filters turn that model into an answerable question.
- Dashboards vs reports — A report is multi-page and tied to one dataset; a dashboard is a single-page canvas of pinned tiles that can span several reports.
- Verifying integration (K2, A5, A6) — Refresh history, data lineage and cross-checking totals against the source are how you prove the integration is actually working.

Lab 12 — Connect, Transform and Model Facilities Data in Power BI

Learning objective: you will be able to connect Power BI to a data source, clean and transform it with Power Query, and build a model with a relationship.

Estimated duration: 60 minutes.

Scenario: Contoso's management wants visibility over facilities spend. You connect Power BI Desktop to the facilities export, use Power Query to clean the deliberately messy data, then relate the requests to the rooms so the model can answer questions neither table could answer alone.

What you will build

A Power BI model with a cleaned Facility Requests table, a Rooms table and a working relationship between them. (Tools used: Power BI Desktop, Power Query Editor.)

Mock data for this lab

- labs/lab-12-powerbi-transform-model/data/facility-requests-export.csv
- labs/lab-12-powerbi-transform-model/data/rooms-export.csv

Step-by-step

133. Open Power BI Desktop. If it is not installed, download it from the Microsoft Store or powerbi.microsoft.com/desktop.

<https://powerbi.microsoft.com/desktop>

134. Select Get data > Text/CSV and open labs/lab-12-powerbi-transform-model/data/facility-requests-export.csv.
135. In the preview select Transform data rather than Load — always inspect before you load.
136. In Power Query, examine the column quality bar. Note the errors and blanks that the export contains.
137. Set the data types: Request Title and Description to Text, Date Requested to Date, Estimated Cost to Decimal Number, Category/Priority/Status to Text.
138. Use Remove rows > Remove blank rows to drop the empty export rows.
139. Select the Estimated Cost column and use Replace values to replace the text 'N/A' with null, then Replace nulls with 0 so the column aggregates cleanly.
140. Use Transform > Format > Trim and then Capitalize Each Word on the Category column to fix the inconsistent casing in the export.
141. Use Remove columns to drop the internal 'ImportBatchId' column — data the report does not need should not reach the model.
142. Rename the query to 'Facility Requests'. Select Home > Close & Apply.
143. Repeat Get data > Text/CSV for rooms-export.csv, set the types, and name the query 'Rooms'.
144. Open Model view. Drag Room from Facility Requests onto Room Name in Rooms to create the relationship, and confirm it is many-to-one with a single cross-filter direction.

145. Verify the model: create a quick table visual showing Building and Sum of Estimated Cost. If it returns values, the relationship works (A6).

Test it

Both tables load with no errors, Estimated Cost is numeric with no nulls, Category values are consistently cased, and a table visual of Building against Sum of Estimated Cost returns non-blank figures.

If it doesn't work

- Estimated Cost will not convert to a number — Text such as 'N/A' remains. Replace it with null first, then change the data type.
- The relationship will not create — The key columns have different data types or trailing spaces. Trim both and confirm both are Text.
- The visual shows blank for every building — The relationship direction is wrong or the key values do not match. Check the room identifiers in both tables are written identically.

Checkpoint questions

- Why transform data in Power Query instead of after loading it?
- What does a many-to-one relationship let you do?
- How would you prove a Power BI report matches its source system?

Lab 13 — Visualise, Publish and Share a Facilities Dashboard

Learning objective: you will be able to build report visuals, publish to the Power BI service, pin a dashboard and share it, then verify the published output against the source.

Estimated duration: 60 minutes.

Scenario: A model is only useful once people can see it. You build the report visuals management asked for, publish to the Power BI service, pin the key visuals to a dashboard, share it, and then verify the published figures match the source data — the final check that the integration works end to end.

What you will build

A published Power BI report and a shared dashboard showing request volume by category, cost by building and a priority breakdown, with verified totals. (Tools used: Power BI Desktop, Power BI service, workspaces, dashboards.)

Step-by-step

146. In Power BI Desktop, on Report view, add a Clustered column chart. Put Category on the X axis and Count of Request Title on the Y axis.

147. Add a Card visual showing Sum of Estimated Cost. Format it as currency with no decimal places.
148. Add a Bar chart with Building on the Y axis and Sum of Estimated Cost on the X axis, sorted descending.
149. Add a Donut chart showing the count of requests broken down by Priority.
150. Add a Slicer for Status so viewers can filter to open requests only.
151. Test the interactivity: select a bar in the Building chart and confirm every other visual cross-filters to that building.
152. Record the unfiltered total of Estimated Cost from the card — you will verify against this figure after publishing.
153. Save the file as 'Contoso Facilities.pbix', then select Publish and choose My workspace.
154. Open the Power BI service, go to My workspace, and open the published report.

<https://app.powerbi.com>

155. Compare the published card total against the figure you recorded. They must match exactly — this is the verification step (A6).
156. Hover each visual and use the pin icon to pin the card, the column chart and the bar chart to a new dashboard named 'Contoso Facilities Dashboard'.
157. Open the dashboard, select Share, and note the sharing options and what a recipient needs — a Power BI licence and access to the workspace.
158. Select the dataset's Settings and note the Refresh section: state how often the data would need to refresh for this dashboard to stay accurate.
159. Explain in one sentence the difference between the report and the dashboard you just created.

Test it

The report shows five working, cross-filtering visuals; the published report's total Estimated Cost matches the figure recorded in Desktop; and the dashboard holds three pinned tiles that open the report when selected.

If it doesn't work

- Publish is greyed out — You are not signed in to Power BI Desktop, or the account has no Power BI licence. Sign in at the top right and retry.
- Published totals differ from Desktop — A filter is applied on the published page, or the file was changed after publishing. Clear filters and re-publish.
- Cannot pin a visual — Pinning is only available in the service, not Desktop. Open the report in app.powerbi.com first.

Checkpoint questions

- What is the difference between a report and a dashboard?
- Why must you verify published totals against the source?
- What does a recipient need in order to open a dashboard you shared?

Topic 06 — Introduction to Power Virtual Agents (Microsoft Copilot Studio)

Agents and topics · knowledge · tools · publishing · troubleshooting integration (LO4 · K4 · A7, A8)

Key concepts

- From Power Virtual Agents to Copilot Studio — Power Virtual Agents is now Microsoft Copilot Studio. The concepts are unchanged — the product builds conversational agents with no code.
- Agents and topics — A topic is one conversation path: trigger phrases start it and a node tree drives the questions, conditions and messages that follow.
- Entities and variables — Entities extract meaning from what the user typed; variables carry those values between nodes and into actions.
- Knowledge sources — Point the agent at SharePoint, a public website or uploaded documents and it answers generatively from that grounded content.
- Tools and actions — An agent calls a Power Automate flow, a connector or an MCP server to do real work — this is where the agent becomes an integration client (K4).
- Publishing and troubleshooting — Publish to Teams, a website or a custom channel; then use test chat, the Topic Checker and analytics to find and fix integration failures (A7, A8).

Lab 14 — Build a Copilot Studio Agent with Topics and Knowledge

Learning objective: you will be able to build a conversational agent, author topics with trigger phrases and conditions, and ground it with a knowledge source.

Estimated duration: 75 minutes.

Scenario: Contoso's facilities inbox is full of the same questions. You build an agent in Copilot Studio, author a topic that collects the request category and priority through a branching conversation, add a knowledge source so the agent answers general questions generatively, and test both in the test chat.

What you will build

A Copilot Studio agent, 'Contoso Facilities Assistant', with a custom topic using a question node and a condition, plus a configured knowledge source. (Tools used: Microsoft Copilot Studio, topics, entities, variables, knowledge sources.)

Mock data for this lab

- labs/lab-14-copilot-studio-agent/data/facilities-faq.md

Step-by-step

160. Open Copilot Studio, confirm the correct environment, and select Create > New agent.

<https://copilotstudio.microsoft.com>

161. Name the agent 'Contoso Facilities Assistant'. Set the description to explain it helps employees raise and track facilities requests, then select Create.

162. Open the Topics tab and review the system topics — Greeting, Escalate, Fallback and End of Conversation come with every agent.

163. Select + Add a topic > From blank. Name it 'Report a Facility Issue'.

164. In the Trigger node add these trigger phrases: 'report an issue', 'something is broken', 'raise a facilities request', 'the printer is broken', 'I need a room set up'. Five or more phrases give the AI enough signal to match reliably.

165. Add a Message node that acknowledges the request: 'I can help you log that. Let me get a few details.'

166. Add a Question node asking 'What kind of issue is it?'. Set Identify to 'Multiple choice options' and add the options Maintenance, Equipment, Supplies and Room Setup. Save the response to a variable named Category.

167. Add a second Question node asking 'How urgent is this?' with the options Low, Medium, High and Urgent, saved to a variable named Priority.

168. Add a Condition node that checks whether Priority is equal to Urgent.

169. On the true branch add a Message node: 'This is urgent — I am flagging it to the facilities team immediately.' On the false branch add: 'Thanks, I have logged this as a standard request.'

170. Save the topic, then open the Test your agent pane and type 'the printer is broken'. Walk the conversation through both the Urgent and the Low path.

171. Open the Knowledge tab and add a knowledge source. Upload labs/lab-14-copilot-studio-agent/data/facilities-faq.md, or point the agent at a public website or SharePoint site.

172. Wait for the source to finish processing, then in the test chat ask a question that is NOT in any topic, such as 'what are the office opening hours?'. The agent should answer generatively from the knowledge source.

173. Note the difference: a topic is an authored, deterministic path; a knowledge source produces a generated answer grounded in content.

Test it

Typing a trigger phrase starts your topic, the two questions branch correctly on Urgent versus Low, and a question outside every topic is answered from the knowledge source rather than falling back.

If it doesn't work

- The topic never triggers — The trigger phrases are too close to another topic's. Add more distinct phrases and re-test; check which topic actually fired in the test pane.
- Knowledge answers are not generated — Generative answers may be disabled. Open Settings > Generative AI and confirm the knowledge source is enabled and finished processing.
- The condition always takes the false branch — The variable is comparing against the wrong value. Confirm the Question node saves to the variable and that the condition compares the same option.

Checkpoint questions

- What is the difference between a topic and a knowledge source?
- What do trigger phrases actually do?
- What is an entity used for in a conversation?

Lab 15 — Add a Tool, Publish the Agent and Resolve Integration Issues

Learning objective: you will be able to call a Power Automate flow from an agent, publish to a channel, then diagnose and resolve integration failures and implement improvements.

Estimated duration: 75 minutes.

Scenario: An agent that only talks is not integrated. You give the agent a tool — a Power Automate flow that writes the request into Dataverse — publish it to a channel, then work through the integration failures that appear when an agent calls a backend system, log each one and implement the fix.

What you will build

An agent that writes a real Facility Request row to Dataverse through a flow, published to a channel, plus a completed integration fault log. (Tools used: Microsoft Copilot Studio, Power Automate, Microsoft Dataverse, publishing channels, analytics.)

Mock data for this lab

- labs/lab-15-agent-publishing-integration/data/integration-fault-log.csv

Step-by-step

174. In your topic, after the Priority question, select + and choose Add a tool > New Power Automate flow. The Power Automate designer opens with the agent trigger and response already in place.
175. In the flow, add inputs to the trigger for Category and Priority as text.
176. Add a Dataverse 'Add a new row' action targeting Facility Requests. Map Request Title to a fixed string plus the Category input, and map Category and Priority from the inputs.
177. In the 'Respond to the agent' action add an output named Result and set it to a confirmation message. Save the flow and name it 'Create Facility Request'.
178. Return to Copilot Studio, refresh the tool list, and select your flow. Map the topic's Category and Priority variables to the flow's inputs.
179. Add a Message node after the tool that returns the flow's Result output to the user.
180. Test it in the test chat. Complete the conversation, then check the Facility Request table in the maker portal for the new row (A4).
181. Now diagnose the common integration failures. Induce the first: remove the mapping for the Priority input, re-test, and record the error the agent returns.

182. Log it in labs/lab-15-agent-publishing-integration/data/integration-fault-log.csv — symptom, component, error text, root cause, fix. This is ability A7.
183. Restore the mapping and re-test to confirm the fix (A8). Then induce a second fault: change the flow's Dataverse action to a table the agent's user cannot access, and record the permission error.
184. Fix the second fault and note the general rule: an agent runs the flow under a connection, so the connection's identity must have rights to the table.
185. Open the Publish tab and select Publish. Then open Channels and review the available options — Teams, a demo website, a custom website, and others.
186. Publish to the Demo website channel, open the generated link, and run one full conversation as an end user would.
187. Open Analytics and review the available measures — session count, engagement rate, resolution rate and the topics that triggered. State how you would use these to identify a topic that needs improvement.
188. Write two improvement recommendations for the agent based on what you observed, and state what you would change to implement each (A8).

Test it

A completed conversation in the published channel creates a real row in the Facility Request table, and your fault log records at least two induced failures with the error text, root cause and the verified fix for each.

If it doesn't work

- The tool does not appear in Copilot Studio — The flow must be in the same environment and saved. Refresh the tool list, and check the flow's solution is in the agent's environment.
- The flow runs but no row is created — A required Dataverse column is unmapped. Open the flow run history and read the Add a new row step's inputs.
- Published agent shows an old version — Changes require a fresh Publish. Publish again and hard-refresh the channel link.

Checkpoint questions

- What is a tool in Copilot Studio, and why does it matter for integration?
- Name two integration failures an agent calling a backend can hit, and their fixes.
- Which channel would you use to put an agent in front of employees at Contoso?

Exam Focus — What PL-900 Actually Tests

PL-900 is a fundamentals exam. It does not ask you to build; it asks you to RECOGNISE which service, app type or flow type fits a described business situation. Study these comparisons until you can pick the right answer from a scenario in a few seconds.

Canvas app vs model-driven app

- Canvas app — you design the screen; pixel control; any connector as a data source; best for task-focused mobile apps. Built from a blank screen or from data.
- Model-driven app — the UI is generated from the Dataverse model; Dataverse only; responsive automatically; best for complex, process-heavy desktop work.
- Scenario cue: 'branded', 'mobile', 'specific layout' points to canvas. 'Manage records', 'complex data model', 'consistent interface' points to model-driven.

Cloud flow vs desktop flow vs business process flow

- Cloud flow — connects cloud services with a trigger and actions; runs in the service; use it whenever a connector or API exists.
- Desktop flow (RPA) — drives the Windows or web user interface like a person; use it ONLY when there is no API, such as a legacy application.
- Business process flow — guides a PERSON through staged work inside a model-driven app; it does not run on its own.
- Scenario cue: 'legacy system with no API' means desktop flow. 'When a row is added' means an automated cloud flow. 'Guide the user through stages' means a business process flow.

The three cloud-flow trigger types

- Automated — an event in a connected service starts it (a row is added, an email arrives).
- Instant — a person starts it manually, usually from a button in Teams or the mobile app.
- Scheduled — a recurrence timer starts it (every Monday at 8am).

Report vs dashboard in Power BI

- Report — multi-page, built on ONE semantic model, fully interactive; authored in Desktop or the service.
- Dashboard — always a SINGLE page of tiles pinned from one or more reports; can only be built in the Power BI service, never in Desktop.

Topics vs knowledge sources in Copilot Studio

- Topic — an authored, deterministic conversation path started by trigger phrases. Use it when the conversation must happen the same way every time, or must take an action.
- Knowledge source — SharePoint, a website or uploaded documents the agent answers from generatively. Use it for the long tail of questions you cannot script.
- Tool — a Power Automate flow, connector, MCP server or agent flow that lets the agent DO something rather than just answer.

Dataverse essentials that are always examined

- A lookup column always lives on the MANY side of a one-to-many relationship.
- A choice column stores an integer VALUE behind its label — flows compare against the value.

- Business rules are no-code and run on every client; use them before reaching for a flow or code.
- Environments isolate dev, test and production; solutions and pipelines move work between them.
- Unmanaged solution = the editable source in Dev. Managed solution = the deployed artefact in Test and Production.

Governance and licensing cues

- A DLP policy groups connectors as Business, Non-business or Blocked, and forbids MIXING groups in one app or flow — this is how data exfiltration is prevented.
- Standard connectors are included with Microsoft 365; premium connectors (Dataverse, SQL Server, HTTP, custom connectors) need a Power Apps or Power Automate licence.
- An on-premises data gateway is required to reach a server inside your own network.
- The three ways to build a custom connector: from blank, from an OpenAPI (Swagger) definition, or from a Postman collection.

Exam Preparation

- First pass: complete every lab in this guide with the portals open in front of you.
- Second pass: redo each lab without the guide until the portal navigation is automatic.
- Learn the comparison tables in the Exam Focus section above — they are the highest-yield revision.
- Practise reading a scenario and naming the right service before you read the options.
- Take the Tertiary Infotech PL-900 practice exam: <https://exams.tertiaryinfotech.com/practice-exams/microsoft/microsoft-pl-900>
- Take the free Microsoft practice assessment, then book the exam through Microsoft Learn: <https://learn.microsoft.com/credentials/certifications/power-platform-fundamentals/>

Glossary

- **Power Platform** — Microsoft's low-code suite: Power Apps, Power Automate, Power BI, Copilot Studio and Power Pages.
- **Dataverse** — The secure, cloud-hosted business data platform underpinning Power Platform — storage plus metadata, security and logic.
- **Table / column / row** — The entity, its fields and its records. Formerly entity / attribute / record.
- **Relationship** — A defined link between two tables — one-to-many, many-to-one or many-to-many.
- **Lookup column** — A column that references a row in another table; it lives on the many side of a 1:N relationship.
- **Choice column** — A column offering a fixed set of options, each storing a label and an integer value.

- **Connector** — A wrapper around an API, exposing it to Power Platform as a set of triggers and actions.
- **Trigger / action** — The event that starts a flow / an operation the flow performs by calling a connector.
- **Canvas app** — A Power App designed screen-first, with pixel control, over any data source.
- **Model-driven app** — A Power App whose interface is generated from the Dataverse data model.
- **Power Fx** — The Excel-like formula language used in canvas apps — SubmitForm, Navigate, Filter, Patch.
- **Cloud flow** — A Power Automate workflow that runs in the service, connecting cloud applications.
- **Desktop flow (RPA)** — A Power Automate flow that automates the Windows or web user interface.
- **Business process flow** — A staged, guided process presented to a user inside a model-driven app.
- **Environment** — An isolated container for apps, flows, data and security within a tenant.
- **DLP policy** — A data loss prevention policy grouping connectors and preventing them being mixed.
- **Solution** — The package that carries customisations between environments; unmanaged in Dev, managed in Production.
- **Pipeline** — Power Platform's built-in mechanism for promoting a solution from Dev to Test to Production.
- **Power Query** — The data transformation engine in Power BI used to clean and reshape data before loading.
- **Semantic model / dataset** — The modelled data a Power BI report is built on.
- **Dashboard** — A single page of tiles pinned from reports in the Power BI service.
- **Copilot Studio** — Microsoft's no-code platform for building conversational agents; formerly Power Virtual Agents.
- **Topic** — One authored conversation path in an agent, started by trigger phrases.
- **Knowledge source** — Content — SharePoint, a website, documents — that an agent answers from generatively.
- **Tool** — A flow, connector, MCP server or agent flow an agent calls to take real action.
- **Channel** — Where a published agent is surfaced — Teams, a website, Microsoft 365 Copilot.
- **AI Builder** — Prebuilt and custom AI models (invoice processing, text recognition) usable in apps and flows.